

```
$ certbot certonly --server http://acme-lan.lan:8000/acme/directory -d db.example.net
```

acme-lan

Real, publicly-trusted certificates for the hosts
the internet can never reach.

No relation to the anvil company. Although both are elaborate schemes that mostly work.

thisisunsafe

I have typed this so many times it stopped being a workaround and became a personality trait.

the motivation, stated honestly

1 One name, two worlds

PUBLIC VIEW

db.example.net

Not reachable. No open ports. Possibly no A record at all.

The CA cannot knock on this door.

LAN VIEW

db.example.net → 192.168.3.5

Split-horizon DNS. A real public domain, pointed at a box that lives behind your firewall forever.

The CA does not need to reach the box. **It only needs proof you control the name.**

That proof is a TXT record. Which means someone has to hold DNS credentials.

dns-01: prove the name, not the network path

2

The three usual answers

OPTION A

DNS token on every host

Every box that needs a cert now holds a credential that can rewrite your whole zone.

Including the printer.

OPTION B

Run your own CA

Genuinely fine — until you have to install that root on every laptop, phone, CI runner and contractor.

Forever.

OPTION C

Self-signed and look away

The industry standard.

See the previous slide.

all three work. none of them made me happy.

3 Why I built it anyway

0

appliances that will
ever run certbot

1

machine holding your
DNS credentials

90

days, renewed forever,
without me

ESXi. iDRAC. The core switch. The NAS. The printer. None of them will ever pip install anything.

the printer has never installed anything. the printer barely prints.

4

The whole idea

Be an ACME server.

Be an ACME client.

Sit in the middle.

CLIENT DIFF

One line.

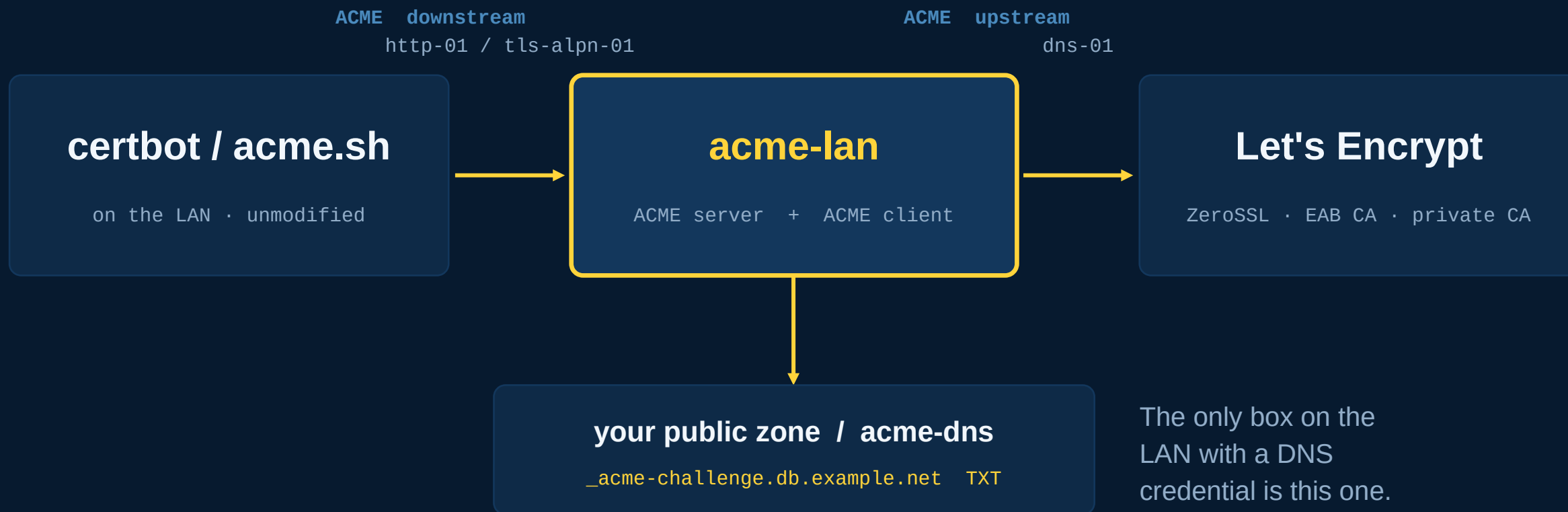
RFC 8555 both directions, so
certbot and acme.sh need no
plugin, no patch and no idea that
anything unusual is happening.

```
- --server https://acme-v02.api.letsencrypt.org/directory  
+ --server http://acme-lan.lan:8000/acme/directory
```

```
# no plugin, no patch, no idea anything unusual is happening
```

5

The architecture, all of it



~1400 lines of FastAPI standing between two ACME conversations

6

The one decision that matters

acme-lan forwards your CSR upstream.

- The issued cert matches the key the client already has
- acme-lan never sees, stores or transmits that private key
- Renewal is just ACME again — no state to keep in sync

```
order = upstream.new_order(csr_from_the_client)    # not one we made up
```

```
# a proxy that generated its own keys would be a very expensive self-signed cert
```


7

Two challenges, two directions

DOWNSTREAM · PROVE IT TO ME

http-01
tls-alpn-01

Port 80 if the box has a web server.

RFC 8737 over port 443 if it does not — the challenge rides in the TLS handshake itself.

Whichever listener the appliance already has.

UPSTREAM · PROVE IT TO THEM

dns-01
http-01 at the edge

dns-01 (default): nothing needs to be publicly reachable. Ever.

edge http-01: faster than DNS propagation, but you need a public IP and a wildcard A record.

One env var switches it.

ACME_LAN_UPSTREAM_CHALLENGE=dns-01

It's always DNS.

The protocol designers agreed with you so completely they made a whole challenge type out of it.

But now one box holds a token that can rewrite your whole zone.

Better than thirty-seven copies of it. Still not something I want to leave lying around the LAN.

8

Chaining to acme-dns

1

Register once

acme-dns hands back a username, a password and a random subdomain.

2

One CNAME, one time

`_acme-challenge.db.example.net`
→ `a1b2c3.auth.example.net`

3

Then never touch DNS again

acme-lan POSTs the TXT value to /update. Removal is a no-op — acme-dns expires them itself.

acme-dns is a DNS server with exactly one opinion, and that opinion is “yes, that TXT record is mine”.

`ACME_LAN_DNS_PROVIDER=acmedns` `# or cloudflare`

9

Blast radius

IF THE ZONE TOKEN LEAKS

Everything

Rewrite any record in the zone. MX. SPF. Your mail. Your website. Certificates for anything you own.

A very bad afternoon.

IF THE ACME-DNS CREDENTIALS LEAK

One TXT record

One delegated subdomain. Certificates for the names you pointed at it — and nothing else.

Bad, but bounded. Your MX records never even hear about it.

Same protocol, one CNAME of separation.

```
# CNAME _acme-challenge.db.example.net -> a1b2c3.auth.example.net
```

10

For the things that will never run ACME

MODE: DEVICE (PREFERRED)

The key never leaves the box

Fetch a CSR the device generated itself, sign it upstream, push back only the certificate.

Nothing secret ever crosses the wire.

MODE: LOCAL

We make the key and push both

For gear that cannot generate a CSR. The key is stored encrypted — never in plaintext at rest — and the UI tells you off about it anyway.

Install happens through deploy plugins: local (write files, run a reload) and ssh (SFTP + reload).
Most network gear is driven over SSH, so a vendor plugin is one class and a list of PluginFields.

ESXi · iDRAC · iLO · switches · printers · NAS · load balancers

11

It comes with a dashboard

Every certificate, every managed host, one page.

Live health badges are a raw TLS probe, not a database lookup.

Certificates link back to the device they were pushed to, and back again.

Expiry warnings, retire-when-done, email + webhook notifications.

The screenshot displays the 'acme-lan' dashboard, an internal ACME server interface. At the top, it shows '4 Certificates issued', '2 Orders total', and '2 Orders valid'. Below this, the 'CERTIFICATES' section lists four entries: 'old-app.lan.test' (unreachable), 'wiki.lan.test' (unreachable), 'esxi01.lan.test' (10d left), and 'db.lan.test' (73d left). Each entry includes a 're-check' link. The 'MANAGED HOSTS' section, with an '+ Add host' button, lists three hosts: 'esxi01' (10d left), 'printer-hp' (none yet), and 'switch-core' (none yet). Each host has 'edit', 'renew', and 'delete' links. At the bottom, the 'PROBE ANY TLS ENDPOINT' section provides a form to test a host's TLS status with fields for Host, Port, and SNI, and a 'Probe' button.

acme-lan
Internal ACME server · certificate dashboard

admin token (if set) [Refresh](#)

4
Certificates issued

2
Orders total

2
Orders valid

CERTIFICATES

DOMAIN(S)	DEVICE	LIVE HEALTH	TRUSTED	EXPIRES	ISSUED	
old-app.lan.test	— ACME client —	unreachable	X	8/24/2026, 11:54:50 AM	8/27/2026, 11:54:50 AM	re-check
wiki.lan.test	— ACME client —	unreachable	X	10/11/2026, 11:54:50 AM	8/27/2026, 11:54:50 AM	re-check
esxi01.lan.test	esxi01	10d left	X	9/7/2026, 11:54:50 AM	8/27/2026, 11:54:50 AM	re-check
db.lan.test	— ACME client —	73d left	X	11/9/2026, 11:54:50 AM	8/27/2026, 11:54:50 AM	re-check

MANAGED HOSTS [+ Add host](#)

Devices that can't run ACME (ESXi, printers, switches), acme-lan issues the cert and pushes it via a deploy plugin.

NAME	DOMAIN(S)	TARGET	PLUGIN	LATEST CERT	LAST STATUS	
esxi01	esxi01.lan.test	192.168.3.5:443	ssh	10d left	deployed	edit renew delete
printer-hp	printer.lan.test	192.168.3.20:443	ssh	none yet	deployed	edit renew delete
switch-core	switch.lan.test	192.168.3.1:443	ssh	none yet	—	edit renew delete

PROBE ANY TLS ENDPOINT

Host: Port: SNI (optional): [Probe](#)

Does a raw TLS handshake — works for LDAPs, SMTPS, and any TLS port, not just HTTPS.

```
# GET /api/certificates · GET /api/hosts · POST /api/health/probe
```

12

Live health, not a spreadsheet

CERTIFICATES

DOMAIN(S)	DEVICE	LIVE HEALTH	TRUSTED	EXPIRES	ISSUED	
old-app.lan.test	— ACME client —	unreachable	✗	8/24/2026, 11:54:50 AM	8/27/2026, 11:54:50 AM	re-check
wiki.lan.test	— ACME client —	unreachable	✗	10/11/2026, 11:54:50 AM	8/27/2026, 11:54:50 AM	re-check
esxi01.lan.test	 esxi01	10d left	✗	9/7/2026, 11:54:50 AM	8/27/2026, 11:54:50 AM	re-check
db.lan.test	— ACME client —	73d left	✗	11/9/2026, 11:54:50 AM	8/27/2026, 11:54:50 AM	re-check

PROBE ANY TLS ENDPOINT

Host: Port: SNI (optional): [Probe](#)

Does a raw TLS handshake — works for LDAPS, SMTPS, and any TLS port, not just HTTPS.

```
{
  "host": "127.0.0.1",
  "port": 8636,
  "reachable": true,
  "error": null,
  "subject": "CN=ldap.lan.test",
  "issuer": "CN=acme-lan demo CA",
  "serial": "290827381d0877212c5ab92e38efa7ea6e3d8157",
  "not_before": "2026-08-26T11:56:14+00:00",
  "not_after": "2026-10-27T11:56:14+00:00",
  "days_remaining": 60,
  "expired": false,
  "self_signed": false,
  "chain_trusted": false,
  "san": [
    "ldap.lan.test"
  ],
  "name_matches": true
}
```

A raw TLS handshake.

Not an HTTP request that assumes port 443 speaks HTTP.

LDAPS. SMTPS. That one appliance on 8443. Anything that completes a handshake gets an honest answer: expiry, chain trust, SAN match, self-signed.

13 Adding a device shouldn't be a JSON blob

Plugins declare their own config.

```
fields: ClassVar[list[PluginField]]
```

The dashboard reads them from the API and renders the real form — filtered by the plugin you picked and the CSR mode you chose.

Write a plugin, get a UI. No frontend involved.

The screenshot shows a web dashboard with a modal for adding a host. The modal is titled "Add host" and contains the following fields:

- Name: esxi01
- Address: 192.168.3.5
- Domains (comma-separated): esxi01.lan
- Port: 443
- Deploy plugin: ssh
- CSR source: device (key stays on device)
- Credential (optional): — none —
- SSH CONFIGURATION:
 - Remote certificate path: /etc/ssl/device.crt
 - Path on the device to upload the certificate chain to.
 - Remote CSR path: /etc/ssl/device.csr
 - Read the device's CSR from this path (or use a CSR command).
 - CSR command: openssl req -new -key /etc/ssl/device.key -subj /CN=host ...
 - Command run on the device that prints a CSR to stdout (alternative to a path).
 - Reload command: /etc/init.d/hostd restart
 - Optional command to run after installing.
 - SSH port: 22

The background shows a list of managed hosts and a JSON blob representing a device configuration:

```
{
  "host": "db.lan.test",
  "port": 443,
  "reachable": false,
  "error": "[Errno -2] Name or service not known",
  "subject": null,
  "issuer": null,
  "serial": null,
  "not_before": null,
  "not_after": null,
  "days_remaining": null,
  "expired": null,
  "self_signed": null,
  "chain_trusted": null,
  "san": null,
  "name_matches": null
}
```

```
# GET /api/deploy-plugins -> the form builds itself
```


14

Should you use this? Probably not.

Hosts reach the internet and can hold DNS creds

certbot + a DNS plugin. Go home.

Happy to run a private CA and push a root everywhere

step-ca / smallstep. Genuinely excellent.

You only need delegated DNS-01

acme-dns on its own. It is the good part of this talk.

One reverse proxy fronts everything you own

Caddy or Traefik. Two lines of config.

Appliances that can't run ACME, split-horizon names, one place to see it all

...fine. That one is this.

a talk that tells you not to use the software is still a talk

15

The Python bits

FastAPI + async SQLAlchemy

One app. RFC 8555 endpoints, REST API, and the SPA, all from one process.

certbot's own acme library

The reference implementation is on PyPI. I did not write an ACME client.

cryptography

CSRs, signing, and the raw TLS probe behind every health badge.

Alembic on startup

The container migrates itself. Upgrades are docker pull.

Pebble + pytest

A real ACME CA in a container that issues deliberately terrible certs. e2e for the whole chain.

Vue 3 + Playwright

The dashboard, and UI tests in CI. Sorry — that bit is not Python.

```
# uv sync && uv run acme-lan
```

16

Three things I'd tell past me

1

Forward the CSR.

Everything else in the design falls out of that one decision.

2

Stay on staging longer.

Rate limits are a patient and thorough teacher.

3

No plaintext keys at rest.

Fernet in the DB, or Key Vault, or Vault. Never a file on disk.

the staging environment exists because I do

Try it

```
docker run -d --name acme-lan -p 8000:8000 -v acme-lan-data:/app/data \
  -e ACME_LAN_EXTERNAL_URL="http://acme-lan.example.net:8000" \
  -e ACME_LAN_SECRET_KEY="$(...fernet key...)" \
  -e ACME_LAN_DNS_PROVIDER="acmedns" \
  ghcr.io/smalljam/acme-lan:latest      # defaults to LE staging
```

github.com/smalljam/acme-lan

MIT · docs for deployment, plugins and key handling in the repo · issues and vendor plugins very welcome

*Thanks. No questions — lightning talk rules — but I'm in the hallway,
and my printer finally has a certificate nobody has to click through.*